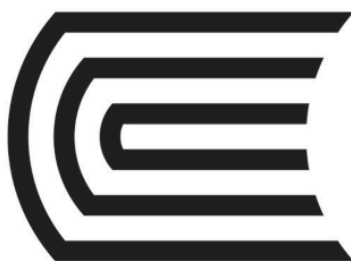


**“ Año de la Esperanza y el Fortalecimiento de la
Democracia ”**



**Universidad
Continental**

Asignatura:
ESTRUCTURA DE DATOS

NRC: 28649

Nro Grupo: 3

Proyecto Grupal
“Desarrollar un Sistema de Gestión de Procesos”

- Integrantes:**
1. Malpartida Arroyo Joseph Rafael
 2. Ñaupari Bonilla Miguel Fabricio
 3. Celis Ricra Daniel Angel
 4. Barrientos Chambergo Miguel Ángel

PERÚ
HUANCAYO - 2026

ÍNDICE

Capítulo 1: Análisis del problema	3
1. Descripción del problema	3
2. Requerimientos del sistema	4
2.1. Requerimientos Funcionales	4
2.2. Requerimientos No Funcionales	4
3. Estructuras de datos propuestas	5
4. Justificación de la elección:	5
Capítulo 2: Diseño de la Solución	6
1. Descripción de estructuras de datos y operaciones:	6
2. Algoritmos principales:	6
2.1. Pseudocódigo para agregar proceso.	6
2.2. Pseudocódigo para cambiar el estado del proceso.	6
3. Diagramas de Flujo:	6
4. Justificación del diseño:	6
Capítulo 3: Solución Final	7
1. Código limpio, bien comentado y estructurado.	7
2. Capturas de pantalla de las ventanas de ejecución con las diversas pruebas de validación	7
3. de datos	7
4. Manual de usuario	7
Capítulo 4 :Evidencias de trabajo colaborativo	8
1. Repositorio y control de versiones	8
1.1. Registro de commits	8
1.2. Historial de ramas	8
1.3. Evidencia por cada integrante del equipo	9
1.3.1. Malpartida Arroyo Joseph Rafael	9
1.3.2. Ñaupari Bonilla Miguel Fabricio	9
1.3.3. Celis Ricra Daniel Angel	9
1.3.4. Barrientos Chambero Miguel A	10
1.4. Enlace a la herramienta colaborativa	10
2. Plan de trabajo y roles asignados	10
2.1. Documento inicial donde se asignan tareas y responsabilidades.	10
2.2. Cronograma con fechas límite para cada entrega parcial.	11
2.3. Registro de reuniones o comunicación del equipo (Actas de reuniones.).	11

Sistema de Gestión de Procesos

Capítulo 1: Análisis del problema

1. Descripción del problema

Hoy en día los sistemas operativos se deben encargar de administrar los múltiples procesos en forma simultánea, es por eso que es necesario que el uso de la memoria y el procesador sean eficientes y ordenados. Sin embargo, realizar esto resulta complicado, ya que la información está en constante cambio durante la ejecución del sistema, por esto se necesitan estructuras dinámicas que permitan gestionar los datos de manera flexible y organizada.

Debido a esta necesidad, se propone desarrollar un Sistema de Gestión de Procesos capaz de simular algunas funciones básicas que realiza un sistema operativo, entre ellas el registro, la búsqueda, eliminación y priorización de procesos, además de la administración de memoria y la planificación de CPU.

Para poder realizar este proyecto se implementarán estructuras de datos dinámicas lineales como listas enlazadas, colas y pilas, las cuales serán desarrolladas manualmente mediante nodos y punteros en el entorno Dev C++, sin utilizar librerías ya predefinidas. De esta manera se busca comprender mejor el funcionamiento de cada estructura y cómo se aplican dentro de un sistema computacional.

Asimismo, este proyecto permitirá mejorar la organización de la información y optimizar el uso de memoria, además ayudará a fortalecer los conocimientos relacionados con estructuras dinámicas, programación modular y manejo de memoria dinámica, aplicando conceptos fundamentales del curso de Estructura de Datos en una situación cercana al funcionamiento real de un sistema operativo.

2. Requerimientos del sistema

2.1. Requerimientos Funcionales

Son las funciones que realizará el sistema.

- El sistema permitirá registrar nuevos procesos.
- También permitirá eliminar procesos redundantes.
- Podrá buscar procesos por ID o por nombre.
- Será posible corregir la prioridad de un proceso.
- El sistema organizará los procesos según su importancia en una cola.
- Además, ejecutará procesos mediante el desencolamiento.
- Mostrará la cola actual de procesos registrados.
- Permitirá asignar memoria usando una pila.
- También liberará memoria cada vez que sea necesario.
- El usuario podrá visualizar el estado actual de la memoria.
- El sistema tendrá una interfaz para interactuar con el usuario.

2.2. Requerimientos No Funcionales

Son las características y condiciones que debe cumplir el sistema.

- El sistema será desarrollado en DEV C++.
- Solo se utilizarán estructuras de datos dinámicas lineales.
- Las estructuras deberán implementarse desde cero, sin librerías predefinidas.
- Cada estructura incluirá operaciones como inserción, eliminación y búsqueda.
- El sistema deberá contar con un mecanismo de almacenamiento permanente de datos.
- El código fuente estará en un repositorio Git.
- El proyecto incluirá documentación en formato PDF.
- El sistema deberá funcionar correctamente durante la demo

3. Estructuras de datos propuestas

En nuestro proyecto se van a utilizar estructuras de datos dinámicas ya que nos permiten manejar la información de manera más flexible y organizada, a diferencia de las estructuras estáticas, las dinámicas no necesitan un tamaño fijo desde el inicio, es por eso que durante la ejecución del programa se podrán agregar o eliminar elementos según sea necesario, gracias a todo esto el sistema podrá adaptarse mejor a las diferentes situaciones que puedan presentarse.

No todas las estructuras funcionan de la misma manera, por eso se utilizarán distintos tipos dependiendo de lo que necesite el sistema. Por ejemplo, se usarán listas porque permiten almacenar y recorrer información de forma ordenada, también se implementarán pilas ya que trabajan con el principio de “último en entrar, primero en salir”, lo cual resulta útil para algunas operaciones dentro del programa. De igual manera, se utilizarán colas porque siguen el orden de “primero en entrar, primero en salir”, ayudando así a mantener una secuencia adecuada en la ejecución de procesos.

Gracias al uso de estas estructuras, el sistema podrá organizar mejor la información y trabajar de una manera más eficiente. Además, permitirá que el programa tenga un mejor control de los datos y pueda responder correctamente ante diferentes casos durante su ejecución.

4. Justificación de la elección:

La lista se utilizará para organizar los procesos porque permite trabajar con información de manera fácil, ya que la cantidad de procesos dentro del sistema puede variar constantemente. Gracias a esta estructura, será más fácil agregar, eliminar, buscar o modificar procesos sin necesidad de mover todos los datos almacenados. La cola ayudará a representar mejor el funcionamiento de un planificador de CPU, permitiendo que los procesos se atiendan no solo por orden de llegada, sino también según su nivel de importancia, mejorando así la ejecución de tareas. La pila servirá para simular la gestión de memoria, ya que trabaja bajo el principio de “último en entrar, primero en salir”, similar a la forma en que muchos sistemas administran recursos durante la ejecución de programas. Los punteros serán importantes para conectar nodos y manejar mejor la memoria, permitiendo crear, modificar y eliminar elementos de una manera más fácil. Además, el programa se desarrollará mediante funciones para tener un código más ordenado, entendible y fácil de mejorar en futuras modificaciones. También se emplearán distintas estructuras de control: el ciclo “while” permitirá recorrer listas, colas y pilas mientras existan nodos disponibles; el “do while” se utilizará principalmente en el menú para asegurar que este se ejecute al menos una vez y el for servirá en procesos donde sea necesario repetir instrucciones una cantidad determinada de veces. De igual manera, la estructura “switch” ayudará a organizar las opciones del menú de una forma más clara, mientras que las condiciones “if” e “if-else” permitirán validar datos y controlar el flujo del programa.

Capítulo 2: Diseño de la Solución

- 1. Descripción de estructuras de datos y operaciones:**
- 2. Algoritmos principales:**
 - 2.1. Pseudocódigo para agregar proceso.**
 - 2.2. Pseudocódigo para cambiar el estado del proceso.**
- 3. Diagramas de Flujo:**
- 4. Justificación del diseño:**

Capítulo 3: Solución Final

1. Código limpio, bien comentado y estructurado.
2. Capturas de pantalla de las ventanas de ejecución con las diversas pruebas de validación
3. de datos
4. Manual de usuario

Capítulo 4 :Evidencias de trabajo colaborativo

1. Repositorio y control de versiones

1.1. Registro de commits

Se corrigió las justificaciones de la elecciones del caso.	Verified	309bfad		
MiguelAngel-art authored 35 minutes ago				
Eliminé un archivo erróneo	Verified	1b03251		
Fab072014 authored 1 hour ago				
Se corrigió y mejoro las estructuras de datos propuestas	Verified	5eda93		
Joseph20175 authored 1 hour ago				
Corrección de requerimientos.	Verified	3987b88		
DanielCelis2121 authored 1 hour ago				
Corrección de los requerimientos.	Verified	caeb9e9		
DanielCelis2121 authored 1 hour ago				
Se corrigió y mejoró el README del proyecto	Verified	3838bbb		
Fab072014 authored 1 hour ago				
Se corrigió y mejoró la descripción del problema	Verified	486b9cb		
Fab072014 authored 1 hour ago				
Commits on May 18, 2026				

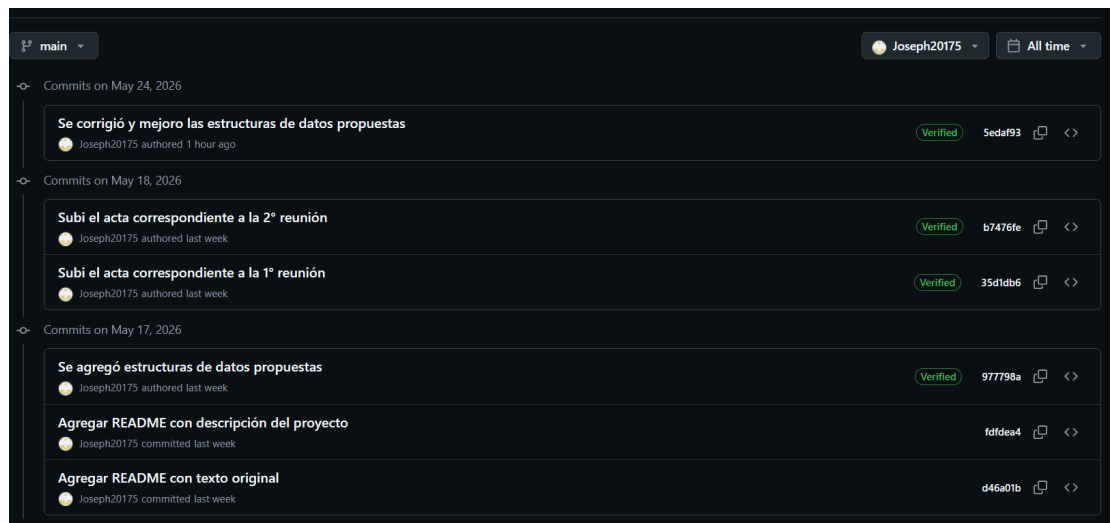
Commits on May 18, 2026				
Subí el acta correspondiente a la 2ª reunión	Verified	b7476fe		
Joseph20175 authored last week				
Subí el acta correspondiente a la 1ª reunión	Verified	35d1db6		
Joseph20175 authored last week				
Commits on May 17, 2026				
Se agregaron las justificaciones de las elecciones del caso	Verified	beef3d6		
MiguelAngel-art authored last week				
Se agregó estructuras de datos propuestas	Verified	977798a		
Joseph20175 authored last week				
Agregar README con descripción del proyecto		fdfdea4		
Joseph20175 committed last week				
Agregar README con texto original		d46a01b		
Joseph20175 committed last week				
Se agrego los requerimientos del caso	Verified	ef5f609		
DanielCelis2121 authored last week				
Realicé descripción del problema	Verified	78e6214		
Fab072014 authored last week				
Estructura inicial: Carpetas Vinformes y Actas		f46488d		
Fab070520 committed last week				

1.2. Historial de ramas

En el proyecto decidimos usar solamente la rama principal (main). La razón fue que el proyecto era simple y el equipo era pequeño. Esto permitirá trabajar de una manera más sencilla.

1.3. Evidencia por cada integrante del equipo

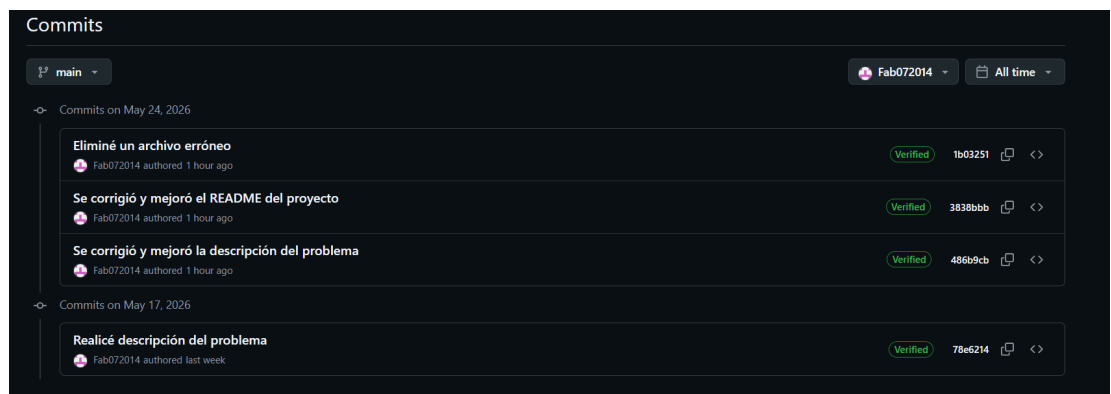
1.3.1. Malpartida Arroyo Joseph Rafael



This screenshot shows the commit history for user Joseph20175 on the 'main' branch. The interface is in dark mode. At the top, there are filters for the user and 'All time'. The commits are grouped by date. On May 24, 2026, there is one commit: 'Se corrigió y mejoro las estructuras de datos propuestas' (Verified, commit 5eda993). On May 18, 2026, there are two commits: 'Subí el acta correspondiente a la 2ª reunión' (Verified, commit b7476fe) and 'Subí el acta correspondiente a la 1ª reunión' (Verified, commit 35d1ab6). On May 17, 2026, there are three commits: 'Se agregó estructuras de datos propuestas' (Verified, commit 977798a), 'Agregar README con descripción del proyecto' (commit fdfdea4), and 'Agregar README con texto original' (commit d46a01b).

Commit Message	Author	Commit Hash	Verified
Se corrigió y mejoro las estructuras de datos propuestas	Joseph20175 authored 1 hour ago	5eda993	Yes
Subí el acta correspondiente a la 2ª reunión	Joseph20175 authored last week	b7476fe	Yes
Subí el acta correspondiente a la 1ª reunión	Joseph20175 authored last week	35d1ab6	Yes
Se agregó estructuras de datos propuestas	Joseph20175 authored last week	977798a	Yes
Agregar README con descripción del proyecto	Joseph20175 committed last week	fdfdea4	No
Agregar README con texto original	Joseph20175 committed last week	d46a01b	No

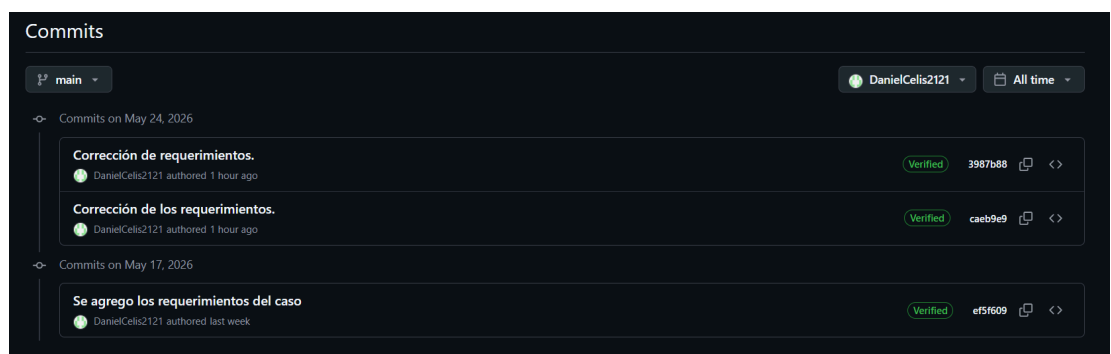
1.3.2. Ñaupari Bonilla Miguel Fabricio



This screenshot shows the commit history for user Fab072014 on the 'main' branch. The interface is in dark mode. At the top, there are filters for the user and 'All time'. The commits are grouped by date. On May 24, 2026, there are three commits: 'Eliminé un archivo erróneo' (Verified, commit 1b03251), 'Se corrigió y mejoró el README del proyecto' (Verified, commit 3838bbb), and 'Se corrigió y mejoró la descripción del problema' (Verified, commit 486b9cb). On May 17, 2026, there is one commit: 'Realicé descripción del problema' (Verified, commit 78e6214).

Commit Message	Author	Commit Hash	Verified
Eliminé un archivo erróneo	Fab072014 authored 1 hour ago	1b03251	Yes
Se corrigió y mejoró el README del proyecto	Fab072014 authored 1 hour ago	3838bbb	Yes
Se corrigió y mejoró la descripción del problema	Fab072014 authored 1 hour ago	486b9cb	Yes
Realicé descripción del problema	Fab072014 authored last week	78e6214	Yes

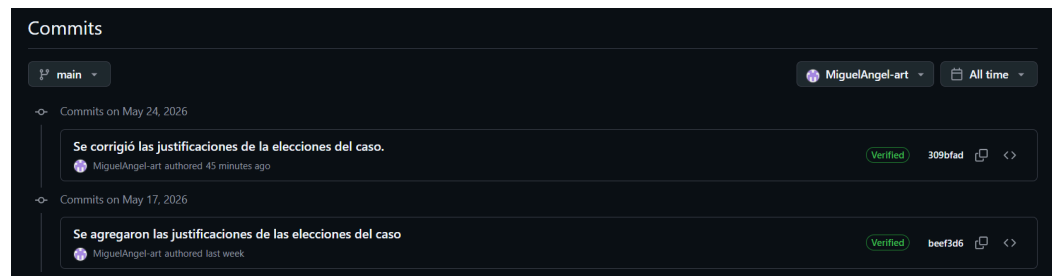
1.3.3. Celis Ricra Daniel Angel



This screenshot shows the commit history for user DanielCelis2121 on the 'main' branch. The interface is in dark mode. At the top, there are filters for the user and 'All time'. The commits are grouped by date. On May 24, 2026, there are two commits: 'Corrección de requerimientos.' (Verified, commit 3987b88) and 'Corrección de los requerimientos.' (Verified, commit caeb9e9). On May 17, 2026, there is one commit: 'Se agrego los requerimientos del caso' (Verified, commit ef5f609).

Commit Message	Author	Commit Hash	Verified
Corrección de requerimientos.	DanielCelis2121 authored 1 hour ago	3987b88	Yes
Corrección de los requerimientos.	DanielCelis2121 authored 1 hour ago	caeb9e9	Yes
Se agrego los requerimientos del caso	DanielCelis2121 authored last week	ef5f609	Yes

1.3.4. Barrientos Chambero Miguel A



1.4. Enlace a la herramienta colaborativa

El siguiente enlace es el de nuestro repositorio:

<https://github.com/Joseph20175/Trabajo-Estructura-de-datos.git>

2. Plan de trabajo y roles asignados

Al principio del proyecto todo el equipo se designaron tareas y responsabilidades para cada integrante, con el fin de trabajar de manera ordenada y avanzar correctamente en todas las etapas del desarrollo del sistema, además se pudo establecer un cronograma con fechas límites para la entrega de cada avance del trabajo, permitiendo controlar mejor el progreso del proyecto.

2.1. Documento inicial donde se asignan tareas y responsabilidades.

- **Malpartida Arroyo Joseph Rafael**
Encargado del desarrollo de la lista enlazada, además de las funciones relacionadas con el registro, búsqueda y eliminación de procesos.
- **Ñaupari Bonilla Miguel Fabricio**
Responsable de la implementación de la cola de prioridad y de la simulación del planificador de CPU dentro del sistema.
- **Celis Ricra Daniel Angel**
Encargado del desarrollo de la pila dinámica para la gestión de memoria y del manejo de persistencia de datos mediante archivos.
- **Barrientos Chambero Miguel Angel**
Responsable del diseño del menú interactivo, integración de módulos y validación general del funcionamiento del sistema.

2.2. Cronograma con fechas límite para cada entrega parcial.

Semana	Actividad	Descripción
Semana 10 18 de Mayo - 22 de Mayo	Capítulo 1: Análisis del problema	Se realizará la descripción del problema, los requerimientos del sistema, las estructuras de datos propuestas y la justificación de las estructuras utilizadas en el proyecto.
Semana 11 25 de Mayo - 29 de Mayo	Capítulo 2: Diseño de la solución	Se desarrollará el diseño del sistema, incluyendo las operaciones principales, pseudocódigos y diagramas de flujo necesarios para el funcionamiento del proyecto.
Semana 12 1 de Junio - 5 de Junio	Capítulo 3: Solución final	Se integrará el código completo del sistema, además se realizarán pruebas de funcionamiento, validación de datos y la elaboración del manual de usuario.

2.3. Registro de reuniones o comunicación del equipo (Actas de reuniones.).

Las actas y registro de las reuniones realizadas en el equipo se encuentran en el siguiente documento: [Actas - Documentos de Google](#)